

Question

A footwear company has N outlets having unique id from 0 to N-1. The outlets are connected using the bi-directional roads directly or via other outlets. There is a cost of stock transfer between the directly connected outlets. Initially, the warehouses were established at K outlets. As the demand of the products increases unexpectedly during the festival seasons, so the stocks need to be transferred quickly among the warehouses. The company has decided to provide the trucks to transport the stock from one warehouse to another but it puts extra overhead on the company. The company has decided to start the truck between the closest warehouses initially. The company wants to find the minimum stock transfer cost between any two warehouses which will be incurred on the company.

Write an algorithm to find the minimum stock transfer cost between any two warehouses which will be overhead on the company.

Input

The first line of the input consists of two space-separated integers - *numOutlets* and *numWarehouses*, representing the number of outlets (N) and the number of warehouses(K)

```
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail
18
19 #include <iostream>
20 // #include <bits/stdc++.h>
21 // #include <unordered_set>
22 using namespace std;
23 int main()
24 {
25     int N, K;
26     cin>>N>>K;
27     cout<<N<<" "<<K;
28     // vector<bool> warehouse(N, 0);
29     int warehouse[501] = {0};
30     for(int i=0; i<K; i++)
31     {
32         int w;
33         cin>>w;
34         warehouse[w] = true;
35     }
36 }
```

Test Cases & Output

RUN SOLUTION

NEXT

0/2 test cases passed.

Console Output :

null

☒ Pre-Defined Test Cases

☐ Custom Test Cases

Case 1



Input:

6 3

Case 2



Input:

7 2

Question

between any two warehouses which will be overhead on the company.

Input

The first line of the input consists of two space-separated integers

- *numOutlets* and *numWarehouses*, representing the number of outlets (N) and the number of warehouses(K).

The next line consists of K space-separated integers- *warehouseID[0]*, *warehouseID[1]*,..., *warehouseID[K-1]*, representing the ids where the warehouses were established initially.

The next line consists of an integer - *directConnection*, representing the number of direct connections among outlets (X).

The next X lines consists of three space-separated integers - A, B, C representing the id of the directly connected outlets (A and B) and the cost of stock transport C between outlet A and B.

Output

Print an integer representing the minimum stock transfer cost between any two warehouses.

Constraints

$0 \leq \text{numWarehouses} \leq \text{numOutlets} \leq 500$

$1 \leq \text{cost}[i][2] \leq 1300$ (i.e., the cost of stock transport between two warehouses is between 1 and 1300)

```
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail
18
19 #include <iostream>
20 // #include <bits/stdc++.h>
21 // #include <unordered_set>
22 using namespace std;
23 int main()
24 {
25     int N, K;
26     cin>>N>>K;
27     cout<<N<<" "<<K;
28     // vector<bool> warehouse(N, 0);
29     int warehouse[501] = {0};
30     for(int i=0; i<K; i++)
31     {
32         int w;
33         cin>>w;
34         warehouse[w] = true;
35     }
36 }
```

Test Cases & Output

RUN SOLUTION

NEXT

0/2 test cases passed.

Console Output :

null

Pre-Defined Test Cases

Custom Test Cases

Case 1



Input:

6 3

Case 2



Input:

7 2

Question

representing the id of the directly connected outlets (A and B) and the cost of stock transport C between outlet A and B.

Output

Print an integer representing the minimum stock transfer cost between any two warehouses.

Constraints

$0 \leq \text{numWarehouses} \leq \text{numOutlets} \leq 500$

$1 \leq \text{cost}[i][2] \leq 1300$ (i.e., the cost of stock transport between two warehouses is between 1 and 1300)

$0 \leq i < \text{numOutlets}$

Example

Input

6 3

1 3 4

7

0 1 10

0 4 7

1 3 12

3 5 3

1 5 10

4 2 2

2 5 3

Output:

8

Explanation:

The warehouses have ids 1, 3, 4. The cost of stock transport between warehouses id 1 and 3 is 12

```
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail
18
19 #include <iostream>
20 // #include <bits/stdc++.h>
21 // #include <unordered_set>
22 using namespace std;
23 int main()
24 {
25     int N, K;
26     cin>>N>>K;
27     cout<<N<<" "<<K;
28     // vector<bool> warehouse(N, 0);
29     int warehouse[501] = {0};
30     for(int i=0; i<K; i++)
31     {
32         int w;
33         cin>>w;
34         warehouse[w] = true;
35     }
36 }
```

Test Cases & Output

RUN SOLUTION

NEXT

0/2 test cases passed.

Console Output :

null

Pre-Defined Test Cases

Custom Test Cases

Case 1



Input:

6 3

Case 2



Input:

7 2

Question

between any two warehouses.

Constraints

$0 \leq \text{numWarehouses} \leq \text{numOutlets} \leq 500$

$1 \leq \text{cost}[i][2] \leq 1300$ (i.e., the cost of stock transport between two warehouses is between 1 and 1300)

$0 \leq i < \text{numOutlets}$

Example

Input

6 3
1 3 4
7
0 1 10
0 4 7
1 3 12
3 5 3
1 5 10
4 2 2
2 5 3

Output:
8

Explanation:
The warehouses have ids 1, 3, 4. The cost of stock transport between warehouses id 1 and 3 is 12, between warehouses id 1 and 4 is 17, and between warehouses id 3 and 4 is 7. So, the minimum cost of stock transport is 8 between warehouse id 3 and 4. So, the output is 8.

```
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail
18
19 #include <iostream>
20 // #include <bits/stdc++.h>
21 // #include <unordered_set>
22 using namespace std;
23 int main()
24 {
25     int N, K;
26     cin>>N>>K;
27     cout<<N<<" "<<K;
28     // vector<bool> wa\\rehouse(N, 0);
29     int warehouse[501] = {0};
30     for(int i=0; i<K; i++)
31     {
32         int w;
33         cin>>w;
34         warehouse[w] = true;
35     }
36 }
```

Test Cases & Output

RUN SOLUTION **NEXT**

0/2 test cases passed.

Console Output :

null

☒ Pre-Defined Test Cases ☐ Custom Test Cases

Case 1	✖		Case 2	✖	
Input:		6 3	Input:		7 2

Question

The current selected programming language is **C++**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `cout` to debug your code. The `cout` may not work in case of syntax/runtime error. The version of **GCC** being used is **5.5.0**.

A footwear company has N outlets having unique id from 0 to $N-1$. The outlets are connected using the bi-directional roads directly or via other outlets. There is a cost of stock transfer between the directly connected outlets. Initially, the warehouses were established at K outlets. As the demand of the products increases unexpectedly during the festival seasons, so the stocks need to be transferred quickly among the warehouses. The company has decided to provide the trucks to transport the stock from one warehouse to another but it puts extra overhead on the company. The company has decided to start the truck between the closest warehouses initially. The company wants to find the minimum stock transfer cost between any two warehouses which will be incurred on the company.

Write an algorithm to find the

```
41 {
42     for(int j=0; j<N; j++)
43         mat[i][j] = -1;
44 }
45
46 int X;
47 cin>>X;
48 for(int i=0; i<X; i++)
49 {
50     int u, v, w;
51     cin>>u>>v>>w;
52     mat[u][v] = w;
53     mat[v][u] = w;
54 }
55
56 int ans=1e9;
57
58 for(int k=0; k<N; k++)
59 {
60     for(int i=0; i<N; i++)
61     {
62         for(int j=0; j<N; j++)
63         {
64             if(i==k || j==k || i==j || mat[i][k] == -1 || mat[k][j]==-1)
65                 continue;
```

Test Cases & Output

RUN SOLUTION

NEXT

Case 1

Input:

```
6 3
1 3 4
7
0 1 10
0 4 7
1 3 12
3 5 3
1 5 10
```

Case 2

Input:

```
7 2
3 6
7
0 1 15
1 2 13
2 3 10
3 4 9
4 5 8
```

Question

The current selected programming language is **C++**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `cout` to debug your code. The `cout` may not work in case of syntax/runtime error. The version of **G++** being used is **5.5.0**.

A footwear company has N outlets having unique id from 0 to $N-1$. The outlets are connected using the bi-directional roads directly or via other outlets. There is a cost of stock transfer between the directly connected outlets. Initially, the warehouses were established at K outlets. As the demand of the products increases unexpectedly during the festival seasons, so the stocks need to be transferred quickly among the warehouses. The company has decided to provide the trucks to transport the stock from one warehouse to another but it puts extra overhead on the company. The company has decided to start the truck between the closest warehouses initially. The company wants to find the minimum stock transfer cost between any two warehouses which will be incurred on the company.

Write an algorithm to find the

```
41 {
42     for(int j=0; j<N; j++)
43         mat[i][j] = -1;
44 }
45
46 int X;
47 cin>>X;
48 for(int i=0; i<X; i++)
49 {
50     int u, v, w;
51     cin>>u>>v>>w;
52     mat[u][v] = w;
53     mat[v][u] = w;
54 }
55
56 int ans=1e9;
57
58 for(int k=0; k<N; k++)
59 {
60     for(int i=0; i<N; i++)
61     {
62         for(int j=0; j<N; j++)
63         {
64             if(i==k || j==k || i==j || mat[i][k] == -1 || mat[k][j]==-1)
65                 continue;
```

Test Cases & Output

RUN SOLUTION

NEXT

Output:

0 4 7
1 3 12
3 5 3
1 5 10
4 2 2
2 5 3

Output of the compiled code will be displayed here. Printing unwanted or ill-formatted data to output will cause the test cases to fail.

Expected Output:

8

Output:

1 2 13
2 3 10
3 4 9
4 5 8
5 6 11
6 0 16

Output of the compiled code will be displayed here. Printing unwanted or ill-formatted data to output will cause the test cases to fail.

Expected Output:

28

Question

The Health Ministry of Torris county has decided to set up medical centers for the benefit of its citizens. The Ministry selects N different cities in the county in which to establish a medical center. Each city is identified by a city ID ranging from 0 to $N-1$. Multiple medicine centers are needed in different cities according to requirement. Each day a medical center will be set up in one of the selected cities. On any two consecutive days the chosen city must be different.

Write an algorithm to determine the maximum number of medicine centers that will be set up in the county covering the maximum number of locations.

Input

The first line of the input consists of an integer - *numCity* representing the number of cities selected for a medical center (N).

The next line consists of N space-separated integers - $loc_1, loc_2, \dots, loc_N$ representing the number of locations to set up a medical centers in a city.

Output

Print an integer representing the maximum number of

```
1 // Sample code to read input and write output:
2
3 /*
4 #include <iostream>
5
6 using namespace std;
7
8 int main()
9 {
10     char name[20];
11     cin >> name;           // Read input from STDIN
12     cout << "Hello " << name; // Write output to STDOUT
13     return 0;
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail
18
19 #include <iostream>
20
21 using namespace std;
22
23 int main()
24 {
25     // Write your code here
26     return 0;
27 }
```

Question

Output

Print an integer representing the maximum number of medicine centers that will be set up in the county covering the maximum number of locations.

Constraints

$$1 \leq numCity \leq 10^4$$

$$1 \leq loq \leq 10^5$$

$$\sum loq \leq 10^5$$

$$1 \leq l \leq N$$

Example

Input:

3
11 4 5

Output:

19

Explanation:

The first, second and third cities are selected with locations 11, 4 and 5, respectively.

The locations of different cities are selected on successive days in a sequence:

1st, 2nd, 1st, 3rd, 1st, 2nd, 1st,
3rd, 1st, 2nd, 1st, 3rd, 1st, 2nd, 1st,
3rd, 1st, 3rd, 1st

So, the maximum number of medical centers that may be set up in the county is 19.

```
1 // Sample code to read input and write output:
2
3 /*
4 #include <iostream>
5
6 using namespace std;
7
8 int main()
9 {
10     char name[20];
11     cin >> name;           // Read input from STDIN
12     cout << "Hello " << name; // Write output to STDOUT
13     return 0;
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail
18
19 #include <iostream>
20
21 using namespace std;
22
23 int main()
24 {
25     // Write your code here
26     return 0;
27 }
```


Question

Output

Print an integer representing the maximum number of medicine centers that will be set up in the county covering the maximum number of locations.

Constraints

$$1 \leq numCity \leq 10^4$$

$$1 \leq loq \leq 10^5$$

$$\sum loq \leq 10^5$$

$$1 \leq l \leq N$$

Example

Input:

3
11 4 5

Output:

19

Explanation:

The first, second and third cities are selected with locations 11, 4 and 5, respectively.

The locations of different cities are selected on successive days in a sequence:

1st, 2nd, 1st, 3rd, 1st, 2nd, 1st, 3rd, 1st, 2nd, 1st, 3rd, 1st, 2nd, 1st, 3rd, 1st, 3rd, 1st

So, the maximum number of medical centers that may be set up in the county is 19.

1 // Sample code to read input and write output:

2

3 /*

4 #include <iostream>

5

6 using namespace std;

7

8 int main()

9 {

10 char name[20];

11 cin >> name; // Read input from STDIN

12 cout << "Hello " << name; // Write output to STDOUT

13 return 0;

14 }

15 */

16

17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail

18

19 #include <iostream>

20

21 using namespace std;

22

23 int main()

24 {

25 // Write your code here

Test Cases & Output

RUN SOLUTION

NEXT

0/2 test cases passed.

Pre-Defined Test Cases

Case 1

Input:

4

2 2 2 2

Output:

Output of the compiled code will be displayed here. Printing unwanted or ill-formatted data to output will cause the test cases to fail.

Expected Output:

8

Custom Test Cases

Case 2

Input:

3

7 2 3

Output:

Output of the compiled code will be displayed here. Printing unwanted or ill-formatted data to output will cause the test cases to fail.

Expected Output:

11

Question

The Cytes Lottery is the biggest lottery in the world. On each ticket, there is a string of a-z letters. The company produces a draw string S. A person wins if his/her ticket string is a special substring of the draw string. A special substring is a substring which can be formed by ignoring at most K characters from drawString. For example, if draw string = "xyzabc" and tickets are [ac zb yhja] with K=1 then the winning tickets will be 2 i.e ac (won by ignoring "b" in drawstring) and zb (won by ignoring "a" in drawstring).

Now, some people change their ticket strings in order to win the lottery. To avoid any kind of suspicion, they can make the following changes in their strings:

1. They can change character 'o' to character 'a' and vice versa.
2. They can change character 't' to character 'l' and vice versa.
3. They can erase a character from anywhere in the string.

Note that they can ignore at most 'K' characters from the draw string to get a match with the ticket string. Write an algorithm to find the number of people who win the lottery (either honestly or by cheating).

Input

The first line of the input consists of

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * ticketList is a list representing the tickets.
10 drawString is a string representing the draw string.
11 tolerance is an integer representing the maximum number of characters that can be deleted.
12 */
13 int winTkt (vector<string> ticketList, string drawString, int tolerance)
14 {
15     int answer;
16     // Write your code here
17
18     return answer;
19 }
20
21
22 int main()
23 {
24
25     //input for ticketList
26     int ticketList_size;
27     cin >> ticketList_size;
28     vector<string> ticketList;
29     for ( int idx = 0; idx < ticketList_size; idx++ )
30     {
31         string temp;
32         cin >> temp;
33         ticketList.push_back(temp);
34     }
```

Question

Input

The first line of the input consists of an integer - *numTickets*, representing the number of tickets (N).

The second line consists of N space-separated strings - *tickets1*, *tickets2*,....., *ticketsN*, representing the tickets.

The third line consists of a string - *drawString*, representing the draw string (S).

The last line consists of an integer - *tolerance*, representing the maximum number of characters that can be deleted from the *drawString* (K).

Output

Print an integer representing the number of winning tickets (either fairly or by cheating).

Constraints

$0 \leq \text{numTickets} \leq 1000$

$0 \leq \text{length of drawString} \leq 200$

$0 \leq \text{length of tickets} \leq 200$

$0 \leq \text{tolerance} \leq 1000$

Note

The *drawString* contains lowercase English alphabets.

Example

Input:

3

abcde aoc actld

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * ticketList is a list representing the tickets.
10 drawString is a string representing the draw string.
11 tolerance is an integer representing the maximum number of characters that can be deleted.
12 */
13 int winTkt (vector<string> ticketList, string drawString, int tolerance)
14 {
15     int answer;
16     // Write your code here
17
18     return answer;
19 }
20
21
22 int main()
23 {
24
25     //input for ticketList
26     int ticketList_size;
27     cin >> ticketList_size;
28     vector<string> ticketList;
29     for ( int idx = 0; idx < ticketList_size; idx++ )
30     {
31         string temp;
32         cin >> temp;
33         ticketList.push_back(temp);
34     }
```

Question

$0 \leq \text{numTickets} \leq 1000$
 $0 \leq \text{length of drawString} \leq 200$
 $0 \leq \text{length of tickets} \leq 200$
 $0 \leq \text{tolerance} \leq 1000$

Note

The *drawString* contains lowercase English alphabets.

Example

Input:

```
3
abcde aoc actld
aabacd
1
```

Output:

```
2
```

Explanation:

For the first ticket "abcde" - delete 'e' from the *tickets* string and delete 'a' from *drawString* (aabacd). So, the new first ticket 'abcd' is a substring of the new *drawString*.

For the second ticket "aoc" - change 'o' to 'a' so that the new string will be "aac" which is a substring of the *drawString*.

For the third ticket "actld" - by applying any operation on the tickets and on the *drawString* we cannot get any substring of the *drawString*.

Therefore, there are only 2 winning tickets.

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * ticketList is a list representing the tickets.
10 drawString is a string representing the draw string.
11 tolerance is an integer representing the maximum number of characters that can be deleted.
12 */
13 int winTkt (vector<string> ticketList, string drawString, int tolerance)
14 {
15     int answer;
16     // Write your code here
17
18
19     return answer;
20 }
21
22 int main()
23 {
24
25     //input for ticketList
26     int ticketList_size;
27     cin >> ticketList_size;
28     vector<string> ticketList;
29     for ( int idx = 0; idx < ticketList_size; idx++ )
30     {
31         string temp;
32         cin >> temp;
33         ticketList.push_back(temp);
34     }
```


Question

The current selected programming language is **C++**. We emphasize the submission of a fully working code over partially correct but efficient code. Once **submitted**, you cannot review this problem again. You can use `cout` to debug your code. The `cout` may not work in case of syntax/runtime error. The version of **G++** being used is **5.5.0**.

The Cycles Lottery is the biggest lottery in the world. On each ticket, there is a string of a-z letters. The company produces a draw string `S`. A person wins if his/her ticket string is a special substring of the draw string. A special substring is a substring which can be formed by ignoring at most `K` characters from `drawString`. For example, if draw string = "xyzabc" and tickets are [ac zb yhja] with `K=1` then the winning tickets will be 2 i.e ac (won by ignoring "b" in drawstring) and zb (won by ignoring "a" in drawstring).

Now, some people change their ticket strings in order to win the lottery. To avoid any kind of suspicion, they can make the following changes in their strings:

1. They can change character 'o' to character 'a' and vice versa.
2. They can change character 't' to character 'l' and vice versa.
3. They can erase a character

```
1 // Header Files
2 #include<iostream>
3 #include<string>
4 #include<vector>
5 using namespace std;
6
7
8 /*
9  * ticketList is a list representing the tickets.
10 drawString is a string representing the draw string.
11 tolerance is an integer representing the maximum number of characters that can be deleted.
12 */
13 int winTkt (vector<string> ticketList, string drawString, int tolerance)
14 {
15     int answer;
16     // Write your code here
17
18
19     return answer;
20 }
21
22 int main()
23 {
24
25     //input for ticketList
```

Test Cases & Output

RUN SOLUTION

SUBMIT ASSESSMENT

Case 1

Input:

5
abcdl aob acld aobocd aabo
aabacdt

Output:

1
Output of the compiled code will be displayed here. Printing unwanted or ill-formatted data to output will cause the test cases to fail.

Expected Output:

5

Case 2

Input:

5
droqwl slingfo oliketss oticelss drawhhhstring
drawstringandatickets

Output:

3
Output of the compiled code will be displayed here. Printing unwanted or ill-formatted data to output will cause the test cases to fail.

Expected Output:

4